

Day 15: Capstone build and demonstrations

Naeemullah Khan

naeemullah.khan@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

October 7, 2026

- ▶ This is the last day of the course. Part 1 of the lecture reviews the course in one view: the trade-offs that each day measured.
- ▶ After Part 1, the lecture time is for the capstone build. Your team builds, and the instructor visits each team.
- ▶ In the lab, each team completes the measurements, gives a demonstration of 10 minutes, and submits the report.
- ▶ A good demonstration shows a working system and defends each decision with a measured number.

After this day you can:

- ▶ Integrate models, devices, messages, and monitoring into one system.
- ▶ Report system performance with measured numbers.
- ▶ Present and defend design decisions.

Part 1 Course summary

Exercise: name one capstone trade-off and the number that decides it

Week 1: TinyML on the XIAO

- 1 Landscape, budgets
- 2 Sensors, data
- 3 Model to the board
- 4 Quantization
- 5 Audio and vision

Week 2: optimization, Raspberry Pi

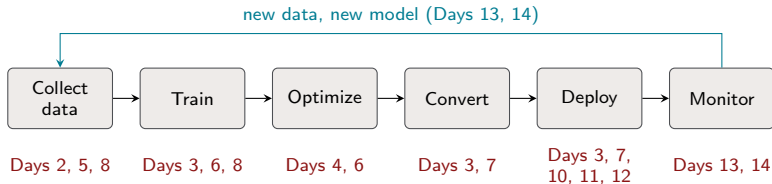
- 6 Pruning, distillation
- 7 Runtimes, graphs
- 8 Object detection
- 9 Benchmarking, energy
- 10 Language models

Week 3: connected systems

- 11 Networks, RTSP
- 12 Decisions, MQTT
- 13 Monitoring
- 14 Production design
- 15 Capstone

- ▶ Week 1 put one model on one microcontroller. Week 2 made models smaller and faster, and measured them. Week 3 joined devices into one system that runs for a long time.
- ▶ Each day measured at least one trade-off. This part puts the measured numbers side by side.

The workflow of Day 1, with its days

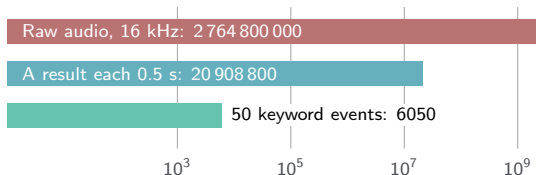


- ▶ Day 9 measured the stages: latency, memory, energy, accuracy.
- ▶ Day 1 said: the device constraints come first. Each later day showed one more constraint: RAM, flash, latency, energy, the network, the time of the people who run the system.
- ▶ The loop does not end at the deployment. A deployed model needs monitoring and updates.

Trade-off	You gain	You pay
Compute on the device	Latency, privacy, data volume	Memory, energy, a smaller model
Fewer bits, fewer weights	Size, often speed	Accuracy, a calibration step
Smaller input	Speed	Small objects
Local decision rules	Fewer false actions	Delay
Reliable delivery	No lost message	Bytes, double messages
Monitoring and logs	Early warning	Storage, false alerts
Security and updates	Safe operation	Bytes, time, work

No row has a free choice. The requirements of the product decide which side wins. The next frames give the measured numbers of each row.

Where to compute: the data that leaves



- ▶ Bytes in one day of one microphone (log scale). Local inference sends results, not raw data: a factor of 132 for the results and of about 457 000 for the events.
- ▶ The radio of the ESP32-S3 needs up to 1122 mW to send; the processor needs 217 mW. Each byte that stays costs no radio energy.
- ▶ The price: the model must fit the device (next frame).

Source: Day 12, Part 3 (data volumes); Day 9 (ESP32-S3 data sheet 2.2, the power is a calculation).

The three budgets of a model

Model	Parameters	MACs	Peak activations
ResNet-18	11 689 512	1 814 073 344	1 003 520
MobileNetV2	3 504 872	300 774 272	1 505 280
Small depthwise CNN	14 186	5 116 160	110 592

- ▶ The parameters fill the flash, the operations take the time and the energy, and the peak activations fill the RAM.
- ▶ The budgets are independent: MobileNetV2 has 3.3 times fewer parameters than ResNet-18, but a 1.5 times larger peak.
- ▶ Of the three, only the small depthwise CNN in `int8` fits a microcontroller with 256 KB of RAM.

Key point: Check all three budgets before you train. A model that misses one budget does not run.

Source: Day 1, lab notebook (calculated, values for one image).

Model	float32	int8
Motion model of Day 3 (file)	26 732 bytes	8680 bytes
MobileNetV2 (accuracy, Imagenette)	77.4 percent	75.2 percent
YOLO11n (mAP50-95, 500 COCO images)	0.424	0.331
Raw window model of Day 3, calibration with one class	100.0 percent	69.2 percent

- ▶ `int8` makes the files 3 to 3.5 times smaller. The loss of accuracy depends on the model: small for a classifier, large for a detector.
- ▶ The calibration data decides the ranges. Data of one class gave a large loss.
- ▶ Quantization-aware training (Day 4) and `int8` weights with `float32` activations reduce the loss.

Source: Day 4 (motion model, calibration); Day 7 (MobileNetV2); Day 8 (YOLO11n). LiteRT on an x86 CPU.

Fashion-MNIST CNN (float32)	File	Accuracy	Time
Baseline, filters 16-32-64	99 320 B	89.1 %	71 μ s
90 percent of the weights set to zero	99 320 B	76.4 %	no gain
Filters removed: 8-16-32	28 408 B	85.6 %	26 μ s
Separable layers	20 872 B	86.3 %	27 μ s

- ▶ Zeros in a dense file save nothing on a normal processor. Remove whole filters, or design the model small.
- ▶ A student trained only on the outputs of a teacher (90.3 percent) reached 85.3 percent: the student needs no labels, and a copy of a model needs only its outputs (Day 14).

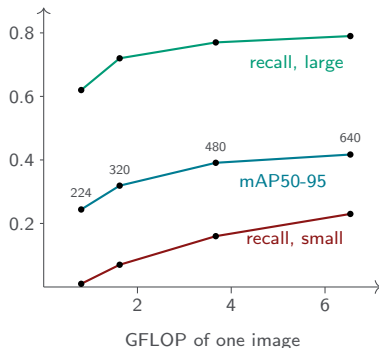
Source: Day 6, Parts 1 to 3 (LiteRT on an x86 CPU, one thread; a time changes by 5 to 10 μ s between two runs).

MobileNetV2, one thread, the same weights	Latency
LiteRT with its reference kernels	362.7 ms
LiteRT with its built-in kernels	41.8 ms
PyTorch	31.9 ms
ONNX Runtime with no graph optimization	26.4 ms
NCNN	16.6 ms
ONNX Runtime	11.7 ms
LiteRT with XNNPACK	11.7 ms

- ▶ A factor of 31 between the slowest and the fastest runtime, with the same model.
- ▶ Fused operators, a good memory layout, and SIMD kernels make the difference (Day 7, Part 3).

Source: Day 7, Part 2 (x86 processor with AVX2, median of 50 runs). The values say nothing about a board.

A smaller input: speed against small objects



- ▶ YOLO11n at 640, 480, 320, and 224 pixels (grey numbers).
- ▶ From 640 to 320 pixels: 4 times fewer operations; the recall of large objects falls from 0.79 to 0.72, of small objects from 0.23 to 0.07.
- ▶ Select the size from the size of the objects in the image, not from the frame rate only.

Source: Day 8, Parts 2 and 3 (500 COCO validation images; recall at the score threshold 0.25).

MobileNetV2, LiteRT, one thread	Value
Mean of the first 5 inferences of a new process	13.35 ms
Median of 2000 inferences after a warm-up	11.621 ms
p99 alone on the core	12.264 ms
p99 with a second program on the core	27.452 ms
95 percent interval of an accuracy, 100 test samples	8.2 points
The same, 1000 test samples	2.6 points

- ▶ Warm up, repeat, and give a distribution: the median and a high percentile.
- ▶ A second program changes the tail, not the median.
- ▶ A small test set cannot show a difference of a few points.

Source: Day 9, Parts 1 and 2 (x86 processor with AVX2).

Duty cycle (light sleep between)	100 %	10 %	1 %	0.1 %
Mean power of the chip	217 mW	22.4 mW	2.95 mW	1.01 mW
Life on 1000 mAh at 3.7 V	17 h	6.9 d	52 d	153 d

- ▶ A faster model saves energy only when the device sleeps after it.
- ▶ The board adds its own rest power: 19 mW in light sleep for the XIAO ESP32S3 Sense, 24 times the chip. With Wi-Fi connected, at least 167.2 mW (Day 14).
- ▶ A Raspberry Pi 5 needs 3.0 to 3.5 W with no load: a mains supply.

Source: Day 9, Part 3 (ESP32-S3 data sheet 2.2, Seeed board values, *Edge AI Engineering*; the life is a calculation).

	llama3.2:1b	llama3.2:3b
Format, file	Q8_0, 1.32 GB	Q4_K_M, 2.02 GB
Generation on a Raspberry Pi 5	8.99 tokens/s	5.3 tokens/s
20 commands to JSON with a schema, correct	18	20

- ▶ Each token reads all weights: the file size times the rate is about the memory bandwidth. Fewer bits give more tokens each second.
- ▶ The larger model was correct more often. The context costs RAM: the cache of the 3B model needs 3.76 GB at 32 768 tokens.
- ▶ A schema and a tool call fix the form of the output, not the facts.

Source: Day 10 (rates: textbook kit lab; JSON test: Ollama on the work computer; sizes: GGUF headers). 1 GB = 10^9 bytes.

Choice	Measured
MJPEG against H.264, 1280 x 720	11.09 against 2.62 Mbit/s
Keyframe each 1 s against each 10 s: wait of a new reader	median 0.50 s against 5.51 s
Reader in order against newest frame, model of 100 ms	age 6711 ms against 56 ms
Default decoder threads against 1 thread	538 ms against 40 ms

- ▶ Compression saves bandwidth and costs start time and CPU time.
- ▶ A slow model must drop frames. A queue in order makes each decision older and older.
- ▶ A default setting can hide a large delay: measure the age of the frame, not only the frame rate.

Source: Day 11, Parts 2 and 3 (MediaMTX, FFmpeg, OpenCV).

Rule on the scores, 60 s	Switch-ons	False	Delay on
One window above 0.5	16.11	8.60	0.04 s
3 of 5 windows	1.07	0.00	0.21 s

MQTT, a link lost for 1.5 s	QoS 0	QoS 1	QoS 2
Bytes of one IMU message	147	193	281
Lost messages	51	0	56
Double messages	0	20	0

- ▶ A rule over several windows removed the false actions for 0.2 s of delay.
- ▶ QoS 1 and a sequence number to drop doubles gave no loss. QoS 2 lost messages with this client after a reconnection.

Source: Day 12, Parts 1 and 2 (simulation, mean of 100 seeds; Mosquitto 2.0.11, paho-mqtt 2.1.0).

Choice	Measured
Alert on the mean of 10 s against 60 s (5 hours)	4 against 1 false alerts
Log each frame against a summary each 60 s (1 hour)	7.89 MB against 18.9 KB
Update of a 9.9 MB folder with restart against resume (100 devices, 7 days)	17 against 0 devices with no file
MQTT with TLS against plain: connect	51.9 ms against 2.4 ms

- ▶ A fast alert is a noisy alert. A longer window gives fewer false alerts and a later alert.
- ▶ A summary with each error keeps the important records for a small part of the storage.
- ▶ Each defense costs bytes or time, but much less than the failure that it prevents.

Source: Day 13, Parts 2 and 3 (simulations); Day 14, Parts 1 and 2 (simulation, Mosquitto on one x86 machine).

1. **Send results, not raw data.** Days 1, 5, 11, 12, 14.
2. **Measure on the target.** On an x86 CPU, `int8` kernels were 1.6 times slower than `float32` (Day 6). On a Raspberry Pi 5, a published `int8` model was 2.9 times faster (Day 7).
3. **A published number is not your number.** Its board, its settings, and its window are different (Day 9).
4. **Check the defaults.** Decoder threads (Day 11), the context length (Day 10), QoS 2 after a reconnection (Day 12), the calibration data (Day 4).
5. **Test the failure.** Cut the link, darken the image, stop the program, fill the disk (Days 12 to 14).
6. **Predict, then measure, then explain the difference.** The method of each lab.

Source: Day 7, Raspberry Pi 5, XNNPACK: 10.84 ms and 3.69 ms.
EdgeML with Raspberry Pi, guide for ExecuTorch.

Form (about 100 words)

The decision. The options. The measured numbers of each option. The trade-off: what you gain and what you pay. The condition that would change the decision.

Example from Day 14, Part 3

We use a window of 2 s. The window is 2.0 s of the alarm budget of 2.6 s, against the requirement of 3.0 s. A window of 1 s would leave more margin, but we did not measure its recall. We keep 2 s until a test on the recordings of the site shows the recall of 1 s.

Your capstone report has a benchmark table and the Decision Log of your main decisions. The demonstration defends them.

Source: Method: *Machine Learning Systems*, instructor guide (assessment). Budget values of the Day 14 example.

Topic	Backup modules of the course
Low power and sleep modes	MC-9
Neural processing units	MC-11 (Grove Vision AI V2), PI-5
Other runtimes	PI-2 (ExecuTorch), GA-4 (llama.cpp), GA-5
Learning on the device	SY-4 (on-device and federated learning)
Security in practice	SY-2 (MQTT with TLS), SY-5, SY-7 (OTA)
Generative applications	GA-1 (retrieval), GA-3 (agents), GA-7 (voice)
Responsible edge AI	SY-6, SY-10 (faults, drift, attacks)

- ▶ Each module has its own deck and lab. Ask the instructor for the material.
- ▶ The textbook (mlsysbook.ai) and the kit labs of the companion books continue each topic.

When	What
Lecture	Parts 2, 3: build; the instructor visits each team
Lab, Part A	Build, measure (80 min)
Lab, Part B	Demonstrations (70 min)
Lab, Part C	Report, feedback (30 min)

A demonstration of 10 minutes

- ▶ The problem and the requirements
 - ▶ The system works: live
 - ▶ The benchmark table: latency, memory, energy
 - ▶ Two decisions and their numbers
 - ▶ Questions
-
- ▶ Finish the measurements before the polish. A working system with numbers scores more than a nice system with no numbers.
 - ▶ Plan B: a recorded input or a video of the system, if the live demonstration fails.

Exercise: One trade-off of your capstone

For your capstone system:

1. Name one trade-off of your design: what you gain and what you pay.
2. Name the number that decides it, the unit, and how you measure it (which board, which window, how many runs or samples).
3. Write the value that would change your decision.

Use the form of the Decision Log. You have 10 minutes. Two teams read their answer to the class.

Answer: One trade-off of your capstone

Example answer for the machine monitor of Day 14:

- ▶ **Trade-off:** a battery on the machine (no cable) against the Wi-Fi link that the dashboard needs.
- ▶ **Number:** the mean current of the XIAO, measured with a USB power meter for 10 minutes in each mode. The budget says at least 167.2 mW with Wi-Fi connected (at most 22 h on 1000 mAh) and 24.2 mW with light sleep and Wi-Fi each hour (6.4 days).
- ▶ **Change:** if the measured mean is below 5.1 mW, a battery of 1000 mAh lasts 30 days and the cable is not necessary.

A good answer names one number, its unit, its method, and a limit.

- ▶ An edge AI system is a set of trade-offs: accuracy against memory, latency, energy, and cost. Each day of the course measured some of them.
- ▶ A design decision needs a measured number and a named trade-off. This is the method of the Decision Log.
- ▶ The capstone joins the parts of the course: a model on the XIAO, a model on the Raspberry Pi, MQTT messages, a local decision, and monitoring.

Key point: Measure before you decide, and write the number next to the decision.

Book chapters

- ▶ *Machine Learning Systems* (mlsysbook.ai), Volume I, chapter “Conclusion”.
- ▶ *Machine Learning Systems*, Volume I, chapters “ML Systems” and “Benchmarking”.
- ▶ *Machine Learning Systems*, Volume II, chapter “Edge Intelligence”.
- ▶ *Edge AI Engineering: Raspberry Pi*, the labs that your capstone uses.

Papers

- ▶ “MLPerf Tiny Benchmark”, Banbury et al., 2021.
- ▶ “MCUNet: Tiny Deep Learning on IoT Devices”, Lin et al., 2020.

- ▶ *Machine Learning Systems* by Vijay Janapa Reddi and contributors, Harvard University. mlsysbook.ai, CC BY-NC-SA 4.0.
- ▶ *Edge AI Engineering: Raspberry Pi* by Marcelo Rovai. github.com/Mjrovai/EdgeML_Made_Ease_ebook and github.com/Mjrovai/EdgeML-with-Raspberry-Pi, GPL-3.0.